Home Work-8

C Programs and Outputs

1) BFS and DFS traversal.

```
#include <stdio.h>

#define N 7

int graph[N][N] = {
    {0,1,1,0,0,0,0}, // A
    {1,0,0,1,1,0,0}, // B
    {1,0,0,0,0,1,0}, // C
    {0,1,0,0,0,0,1}, // D
    {0,1,0,0,0,1,0}, // E
    {0,0,1,0,1,0,0}, // F
    {0,0,0,1,0,0,0}  // G
};

int visited[N];

void bfs(int start) {
    int queue[N], front = 0, rear = 0;
    int v, i;

    for (i = 0; i < N; i++)
        visited[i] = 0;

    queue[rear++] = start;
    visited[start] = 1;

    while (front < rear) {
        v = queue[front++];
        printf("%c ", 'A' + v);

        for (i = 0; i < N; i++) {
            if (graph[v][i] && !visited[i]) {
                queue[rear++] = i;
                visited[i] = 1;
            }
        }
    }
}

void dfs(int v) {
    int i;
    visited[v] = 1;
    printf("%c ", 'A' + v);

    for (i = 0; i < N; i++)
        if (graph[v][i] && !visited[i])
            dfs(i);
}

int main(void) {
    int i;

    printf("BFS: ");
    bfs(1);  // Start at B

    for (i = 0; i < N; i++)
        visited[i] = 0;

    printf("\nDFS: ");
    dfs(1);

    printf("\n");
    return 0;
}
```

Output:

```
BFS: B A D E C G F
DFS: B A C F E D G
```

2) Minimum spanning tree using Prim’s and Kruskal’s algorithms.

```
#include <stdio.h>

#define V 5
#define E 7
#define INF 99999

typedef struct {
    int u, v, w;
} Edge;

Edge edges[E] = {
    {0,1,1}, {0,2,7}, {1,2,5},
    {1,3,4}, {1,4,3}, {2,4,6},
    {3,4,2}
};

void prim(void) {
    int cost[V][V] = {0};
    int selected[V] = {0};
    int i, j, count = 0, total = 0;

    for (i = 0; i < E; i++) {
        cost[edges[i].u][edges[i].v] = edges[i].w;
        cost[edges[i].v][edges[i].u] = edges[i].w;
    }

    selected[0] = 1;
    printf("Prim MST:\n");

    while (count < V - 1) {
        int min = INF, u = -1, v = -1;

        for (i = 0; i < V; i++)
            if (selected[i])
                for (j = 0; j < V; j++)
                    if (!selected[j] && cost[i][j] &&
                        cost[i][j] < min) {
                        min = cost[i][j];
                        u = i;
                        v = j;
                    }

        if (u == -1) {
            printf("Graph is disconnected\n");
            return;
        }

        printf("%c-%c : %d\n",
               'A'+u, 'A'+v, min);
        selected[v] = 1;
        total += min;
        count++;
    }

    printf("Total weight = %d\n", total);
}

int parent[V];

int find(int x) {
    while (parent[x] != x)
        x = parent[x];
    return x;
}

void kruskal(void) {
    int i, j, total = 0, count = 0;
    Edge temp;

    for (i = 0; i < V; i++)
        parent[i] = i;

    for (i = 0; i < E-1; i++)
        for (j = 0; j < E-1-i; j++)
            if (edges[j].w > edges[j+1].w) {
                temp = edges[j];
                edges[j] = edges[j+1];
                edges[j+1] = temp;
```

```
            }

    printf("\nKruskal MST:\n");

    for (i = 0; i < E && count < V-1; i++) {
        int a = find(edges[i].u);
        int b = find(edges[i].v);

        if (a != b) {
            parent[a] = b;
            printf("%c-%c : %d\n",
                   'A'+edges[i].u,
                   'A'+edges[i].v,
                   edges[i].w);
            total += edges[i].w;
            count++;
        }
    }

    printf("Total weight = %d\n", total);
}

int main(void) {
    prim();
    kruskal();
    return 0;
}
```

Output:

```
Prim MST:
A-B : 1
B-E : 3
E-D : 2
B-C : 5
Total weight = 11
Kruskal MST:
A-B : 1
D-E : 2
B-E : 3
B-C : 5
Total weight = 11
```

3) Minimum spanning tree for a weighted graph.

```
#include <stdio.h>

#define V 9
#define E 14

struct Edge {
    int src, dest, weight;
};

struct Edge edges[E] = {
    {0,1,4}, {0,2,8}, {1,2,7},
    {1,3,6}, {1,4,10}, {2,3,2},
    {2,5,9}, {3,4,5}, {3,6,7},
    {4,5,3}, {4,7,8}, {5,7,2},
    {6,7,1}, {6,8,10}
};

int parent[V];

int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]);
    return parent[x];
}

void sortEdges(void) {
    int i, j;
    struct Edge temp;

    for (i = 0; i < E - 1; i++) {
        for (j = 0; j < E - 1 - i; j++) {
            if (edges[j].weight > edges[j + 1].weight) {
                temp = edges[j];
                edges[j] = edges[j + 1];
                edges[j + 1] = temp;
            }
        }
    }
}

void kruskal(void) {
    int i, count = 0, total = 0;

    for (i = 0; i < V; i++)
        parent[i] = i;

    sortEdges();

    printf("Edges in MST:\n");

    for (i = 0; i < E && count < V - 1; i++) {
        int u = find(edges[i].src);
        int v = find(edges[i].dest);

        if (u != v) {
            printf("%d - %d : %d\n",
                   edges[i].src,
                   edges[i].dest,
                   edges[i].weight);

            parent[u] = v;
            total += edges[i].weight;
            count++;
        }
    }

    if (count == V - 1)
        printf("Total MST weight = %d\n", total);
    else
        printf("Graph is disconnected\n");
}

int main(void) {
    kruskal();
    return 0;
}
```

Output:

```
Edges in MST:
6 - 7 : 1
2 - 3 : 2
5 - 7 : 2
4 - 5 : 3
0 - 1 : 4
3 - 4 : 5
1 - 3 : 6
6 - 8 : 10
Total MST weight = 33
```

4) Dijkstra’s shortest path algorithm.

```
#include <stdio.h>

#define V 5
#define INF 99999

int main(void) {
    int g[V][V] = {
        {0,4,2,0,0},
        {4,0,1,5,0},
        {2,1,0,8,0},
        {0,5,8,0,0},
        {0,0,0,0,0}
    };

    /* Add a connection from D to E for a
       connected example. */
    g[3][4] = 2;
    g[4][3] = 2;

    int dist[V], visited[V] = {0};
    int i, j, u;

    for (i = 0; i < V; i++)
        dist[i] = INF;

    dist[0] = 0;

    for (i = 0; i < V; i++) {
        int min = INF;
        u = -1;

        for (j = 0; j < V; j++)
            if (!visited[j] && dist[j] < min) {
                min = dist[j];
                u = j;
            }

        if (u == -1)
            break;

        visited[u] = 1;

        for (j = 0; j < V; j++)
            if (g[u][j] > 0 && !visited[j] &&
                dist[u] + g[u][j] < dist[j])
                dist[j] = dist[u] + g[u][j];
    }

    printf("Shortest distances from A:\n");

    for (i = 0; i < V; i++) {
        if (dist[i] == INF)
            printf("%c : Unreachable\n", 'A'+i);
        else
            printf("%c : %d\n", 'A'+i, dist[i]);
    }

    return 0;
}
```

Output:

```
Shortest distances from A:
A : 0
B : 3
C : 2
```

```
D : 8
E : 10
```

5) Topological sorting of a DAG.

```
#include <stdio.h>

#define V 6

int main(void) {
    int g[V][V] = {0};
    int indegree[V] = {0};
    int queue[V], front = 0, rear = 0;
    int i, j, count = 0, u;

    /* A=0, B=1, C=2, D=3, E=4, F=5 */
    g[0][2] = 1;  // A -> C
    g[1][2] = 1;  // B -> C
    g[1][4] = 1;  // B -> E
    g[2][3] = 1;  // C -> D
    g[4][3] = 1;  // E -> D
    g[3][5] = 1;  // D -> F

    for (i = 0; i < V; i++)
        for (j = 0; j < V; j++)
            if (g[i][j])
                indegree[j]++;

    for (i = 0; i < V; i++)
        if (indegree[i] == 0)
            queue[rear++] = i;

    printf("Topological order: ");

    while (front < rear) {
        u = queue[front++];
        printf("%c ", 'A'+u);
        count++;

        for (j = 0; j < V; j++) {
            if (g[u][j]) {
                indegree[j]--;
                if (indegree[j] == 0)
                    queue[rear++] = j;
            }
        }
    }

    if (count != V)
        printf("\nGraph contains a cycle");
    else
        printf("\n");

    return 0;
}
```

Output:

```
Topological order: A B C E D F
```

6) Hash table using linear probing Insert 76, 93, 40, 47, 10, 55 into a hash table of size 7.

```
#include <stdio.h>

#define SIZE 7

int main(void) {
    int table[SIZE];
    int keys[] = {76, 93, 40, 47, 10, 55};
    int n = 6, i, j, index;

    for (i = 0; i < SIZE; i++)
        table[i] = -1;

    for (i = 0; i < n; i++) {
        index = keys[i] % SIZE;

        for (j = 0; j < SIZE; j++) {
            int pos = (index + j) % SIZE;

            if (table[pos] == -1) {
                table[pos] = keys[i];
                break;
            }
        }

        if (j == SIZE)
            printf("Table full: %d\n", keys[i]);
    }

    printf("Index\tValue\n");

    for (i = 0; i < SIZE; i++)
        if (table[i] == -1)
            printf("%d\tEmpty\n", i);
        else
            printf("%d\t%d\n", i, table[i]);

    return 0;
}
```

Output:

```
Index■Value
0■47
1■55
2■93
3■10
4■Empty
5■40
6■76
```

7) Hash table using quadratic probing

Insert 22, 30, 50 into a hash table of size 7.

First hash function: h(k)=k mod 7

h(k)=kmod7

Collision resolution: (h(k)+i2) mod 7

(h(k)+i2)mod7

```
#include <stdio.h>

#define SIZE 7

int main(void) {
    int table[SIZE];
    int keys[] = {22, 30, 50};
    int n = 3, i, j, pos;

    for (i = 0; i < SIZE; i++)
        table[i] = -1;

    for (i = 0; i < n; i++) {
        int h = keys[i] % SIZE;

        for (j = 0; j < SIZE; j++) {
```

```
            pos = (h + j * j) % SIZE;

            if (table[pos] == -1) {
                table[pos] = keys[i];
                break;
            }
        }

        if (j == SIZE)
            printf("Cannot insert %d\n", keys[i]);
    }

    for (i = 0; i < SIZE; i++) {
        if (table[i] == -1)
            printf("%d : Empty\n", i);
        else
            printf("%d : %d\n", i, table[i]);
    }

    return 0;
}
```

Output:

```
0 : Empty
1 : 22
2 : 30
3 : Empty
4 : Empty
5 : 50
6 : Empty
```

8) Hash table using double hashing

Insert 27, 43, 69, 2, 72 into a hash table of size 7.

h1(k)=k mod 7

h1■(k)=kmod7

h2(k)=1+(k mod 5)

h2■(k)=1+(kmod5)

```
#include <stdio.h>

#define SIZE 7

int main(void) {
    int table[SIZE];
    int keys[] = {27, 43, 69, 2, 72};
    int n = 5, i, j, pos;

    for (i = 0; i < SIZE; i++)
        table[i] = -1;

    for (i = 0; i < n; i++) {
        int h1 = keys[i] % SIZE;
        int h2 = 1 + keys[i] % 5;

        for (j = 0; j < SIZE; j++) {
            pos = (h1 + j * h2) % SIZE;

            if (table[pos] == -1) {
                table[pos] = keys[i];
                break;
            }
        }

        if (j == SIZE)
            printf("Cannot insert %d\n", keys[i]);
    }

    for (i = 0; i < SIZE; i++) {
        if (table[i] == -1)
            printf("%d : Empty\n", i);
        else
            printf("%d : %d\n", i, table[i]);
    }
    return 0;
```

```
}
```

Output:

```
0 : Empty
1 : 43
2 : 2
3 : Empty
4 : 69
5 : 72
6 : 27
```

9) Quadratic probing when the table size is prime.

```
#include <stdio.h>

#define SIZE 7

int main(void) {
    int table[SIZE];
    int keys[] = {22, 30, 50};
    int i, j, pos;

    for (i = 0; i < SIZE; i++)
        table[i] = -1;

    for (i = 0; i < 3; i++) {
        int h = keys[i] % SIZE;

        for (j = 0; j < SIZE; j++) {
            pos = (h + j * j) % SIZE;

            if (table[pos] == -1) {
                table[pos] = keys[i];
                break;
            }
        }

        if (j == SIZE)
            printf("Insertion failed: %d\n", keys[i]);
    }

    printf("Index\tValue\n");

    for (i = 0; i < SIZE; i++) {
        if (table[i] == -1)
            printf("%d\tEmpty\n", i);
        else
            printf("%d\t%d\n", i, table[i]);
    }

    return 0;
}
```

Output:

```
Index■Value
0■Empty
1■22
2■30
3■Empty
4■Empty
5■50
6■Empty
```

10) C program for hash table using separate chaining.

```
#include <stdio.h>
#include <stdlib.h>

#define SIZE 7

struct Node {
    int data;
    struct Node *next;
};

struct Node *table[SIZE];

void initialize(void) {
    int i;

    for (i = 0; i < SIZE; i++)
        table[i] = NULL;
}

int hash(int key) {
    return ((key % SIZE) + SIZE) % SIZE;
}

void insert(int key) {
    int index = hash(key);
    struct Node *p = table[index];

    while (p != NULL) {
        if (p->data == key) {
            printf("Key already exists\n");
            return;
        }
        p = p->next;
    }

    struct Node *newNode =
        malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed\n");
        return;
    }

    newNode->data = key;
    newNode->next = table[index];
    table[index] = newNode;
}

void search(int key) {
    struct Node *p = table[hash(key)];

    while (p != NULL) {
        if (p->data == key) {
            printf("Key found\n");
            return;
        }
        p = p->next;
    }

    printf("Key not found\n");
}

void deleteKey(int key) {
    int index = hash(key);
    struct Node *p = table[index];
    struct Node *prev = NULL;

    while (p != NULL && p->data != key) {
        prev = p;
        p = p->next;
    }

    if (p == NULL) {
        printf("Key not found\n");
        return;
    }

    if (prev == NULL)
```

```
        table[index] = p->next;
    else
        prev->next = p->next;

    free(p);
    printf("Key deleted\n");
}

void display(void) {
    int i;
    struct Node *p;

    for (i = 0; i < SIZE; i++) {
        printf("%d: ", i);
        p = table[i];

        while (p != NULL) {
            printf("%d -> ", p->data);
            p = p->next;
        }

        printf("NULL\n");
    }
}

void cleanup(void) {
    int i;
    struct Node *p, *temp;

    for (i = 0; i < SIZE; i++) {
        p = table[i];

        while (p != NULL) {
            temp = p;
            p = p->next;
            free(temp);
        }

        table[i] = NULL;
    }
}

int main(void) {
    initialize();

    insert(22);
    insert(30);
    insert(50);
    insert(43);

    printf("Hash table:\n");
    display();

    printf("\nSearching 30:\n");
    search(30);

    printf("\nDeleting 30:\n");
    deleteKey(30);

    printf("\nHash table after deletion:\n");
    display();

    cleanup();
    return 0;
}
```

Output:

```
Hash table:
0: NULL
1: 43 -> 50 -> 22 -> NULL
2: 30 -> NULL
3: NULL
4: NULL
5: NULL
6: NULL
Searching 30:
Key found
Deleting 30:
Key deleted
```

```
Hash table after deletion:
0: NULL
1: 43 -> 50 -> 22 -> NULL
2: NULL
3: NULL
4: NULL
5: NULL
6: NULL
```